Clustering and Initialization

Francesco Vago

April 2026

1 Introduction

Clustering is one of the fundamental problems in unsupervised machine learning. Given a set of n data points in $\mathbb{R}^d$, the goal is to partition them into k groups such that points within the same group are more similar to each other than to points in other groups. In the k-means formulation, this is expressed as the minimization of the potential function

$$\phi = \sum_{x \in X} \min_{c \in C} \|x - c\|^2,$$

where $C = \{c_1, \ldots, c_k\}$ is the set of cluster centres. This objective, also called *inertia*, measures the total squared distance between each point and its nearest centre.

Finding the exact solution to this problem is NP-hard, even for $k = 2$. The most widely used approach in practice is Lloyd's algorithm, commonly referred to simply as `k-means`. The algorithm starts from k initial centres, typically chosen uniformly at random from the data points, and then alternates between two steps:

1. assigning each point to its nearest centre;
2. recomputing each centre as the mean of the points assigned to it.

Since both steps are guaranteed to decrease ϕ, the algorithm converges to a local minimum. However, the quality of this local minimum depends heavily on the choice of the initial centres. Two runs of the same algorithm on the same data can produce very different clusterings, and in the worst case the resulting potential can be arbitrarily far from the optimum.

Arthur and Vassilvitskii [1] addressed this limitation by proposing `k-means++`, a simple modification to the initialization step. Instead of choosing centres uniformly at random, `k-means++` selects them with probability proportional to the squared distance from the nearest already-chosen centre (known as D^2-weighting). Formally, the first centre c_1 is chosen uniformly at random from the data, and each subsequent centre c_i is chosen as $x' \in X$ with probability

$$\frac{D(x')^2}{\sum_{x \in X} D(x)^2},$$

where $D(x)$ denotes the distance from x to the nearest centre already selected. After choosing the k centres, the algorithm continues with the standard Lloyd iterations. The authors proved that this initialization alone guarantees $E[\phi] \leq 8(\ln k + 2)\,\phi_{\mathrm{OPT}}$, and that this bound is tight up to a constant factor. Furthermore, they showed that on well-separated datasets the algorithm achieves $O(1)$-competitive performance, complementing an independent result by Ostrovsky et al.

The aim of this project is to implement both `k-means` and `k-means++` from scratch and to empirically evaluate the impact of initialization on clustering performance. Following the experimental setup of the original paper, we compare the two algorithms across multiple trials on both synthetic and real-world datasets, measuring inertia and variability. We also investigate the behaviour of the algorithms under two challenging conditions: high-dimensional data and overlapping clusters.

2 Methodology

2.1 The k-means algorithm

1. Arbitrarily choose k initial centres $\mathcal{C} = \{c_1, ..., c_k\}$.
2. Compute the distance between every centre c_i, for each $i \in \{1, ..., k\}$ for each point. Assign each point p to cluster C_i such that the distance between p and c_i is smaller than the distance between p and any other centre c_j with $j \in 1, ..., k$ and $j \neq i$.
3. Update each centre c_i with the mean of the points in C_i: $c_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$, for all $i \in \{1, .., k\}$
4. Repeat step 2 and 3 until $\mathcal{C}$ no longer changes.

2.2 The k-means++ algorithm

The k-means++ algorithm provides an initialization for the initials centres that improves the result of k-means with simple random chosen centres from the points. After this initialization the algorithm proceeds as standard k-means. The initialization procedure is described below:

1. Choose an initial centre c_1 uniformly at random from $\mathcal{X}$
2. Choose the next centre c_i, selecting $c_i = x' \in \mathcal{X}$ with probability $\frac{D(x')^2}{\sum_{x \in \mathcal{X}} D(x)^2}$.
3. Repeat step 2 until k centres have been chosen.
4. Proceed with standard k-means algorithm.

Where $D(x)$ denotes the shortest distance from x to the nearest already chosen centre Both algorithms have been implemented in Python with NumPy for numerical computation. The distance computation has been vectorized by exploiting the expansion of the squared Euclidean norm:

$$\|x_i - c_j\|^2 = \|x_i\|^2 - 2\, x_i \cdot c_j + \|c_j\|^2$$

Given the data matrix $X \in \mathbb{R}^{n \times d}$ and the centroid matrix $C \in \mathbb{R}^{k \times d}$, this yields the $n \times k$ matrix of squared distances via a single matrix multiplication and broadcasting, avoiding the construction of the $n \times k \times d$ intermediate array that a direct computation would require. The element-wise maximum with zero is applied to prevent negative values arising from floating-point errors. Since the squared distance preserves the ordering, the square root is unnecessary for the assignment step.

Some important details about implementation are that for k-means the initial centres have been chosen uniformly at random from the data points, the special case of an empty cluster has been handled by choosing another centre, in particular by selecting the point with maximum distance from its centre. Finally the convergence of the algorithm was checked using the function `numpy.allclose` that considers the floating point error margin.

2.3 Datasets

The algorithms k-means and k-means++ have been tested on three datasets, following the experimental setup of [1]. The first dataset is a synthetic dataset, which aims to reproduce *Norm25*. This dataset was obtained by generating 25 real centres uniformly drawn at random from a 15 dimensional hypercube with side of length 500. For each centre 100 points were drawn from a Gaussian of mean equal to the point and standard deviation of 1. This procedure produced a dataset containing 25 well separated clusters.

The algorithms have been tested also on real world dataset, in particular on the *Cloud* dataset (1024 points in 10 dimensions) and *Spam* dataset (4601 points in 58 dimensions). The *Intrusion* dataset was not tested due to its large dimension and hardware limitations.

2.4 Metrics

For each dataset both k-means and k-means++ have been run for 50 trials for values of $k = 10,\ 25,\ 50$. These particular values were taken from [1]. For reasons of reproducibility every trial used as random seed its index, starting from 0 to 49. For each trial, two quantities were measured: the inertia and the variability of the clustering outcome. In particular *inertia* was measured as

$$\sum_{i=1}^{k} \sum_{x \in C_i} ||x - c_i||^2$$

where C_i is the i-th cluster and c_i is the associated centroid. The following metrics are reported: mean, standard deviation, minimum, maximum and median of the inertia measured in the 50 trials. For variability it was measured standard deviation of inertia, and the coefficient of variation of inertia CV = $\frac{\sigma}{\mu}$.

Finally the extensions used two more datasets that were obtained with the same function used to replicate *Norm25* but with different parameters. The high dimensionality extension used a synthetic dataset obtained using 25 true centres, 100 points per centre uniformly drawn at random from a 450 dimension hypercube with side of 500, while for the overlapping clusters extension a synthetic dataset obtained using 25 true centres, 100 points per centre randomly drawn from a Gaussian with standard deviation of 2 uniformly drawn at random from a 15 dimension hypercube with side of 20 was used.

3 Results

This section presents the experimental results obtained by running k-means and k-means++ on the datasets described above. For each dataset and each value of k, summary statistics of the inertia over the 50 trials are reported, along with some representative plots. Additional statistics and plots can be found in the Python notebook. Base experiments results are discussed first, followed by extensions.

3.1 Synthetic dataset

This dataset is the most representative dataset among all the tested ones. This is because it was specifically created for having well separated clusters, leading to explicative results.

	Mean ϕ			CV	
k	KM	KM++	Impr.	KM	KM++
10	336881380.53	317794775.30	5.67%	0.07	0.06
25	101332693.28	37032.77	99.96%	0.32	0.00
50	13236496.21	34581.35	99.74%	0.97	0.00

Table 1: Results on the synthetic dataset. The percentage represent the improvement of k-means++ with respect to k-mean, computed with $100 \cdot \left(1 - \frac{\text{k-means++ value}}{\text{k-means value}}\right)$ like in the paper.

Table 3.1 shows a direct comparison between the results of k-means and k-means++. It is clear that the k-means++ significantly reduces the average inertia over the 50 trials, in the cases of $k = 25$ and $k = 50$, with very small variability which means that the clustering problem is typically solved better with k-means++. By observing the plots, specifically the histogram of inertia it can be seen that the bins of k-means are more sparse while the bins of k-means++ are more concentrated to the left, which means lower inertia with lower variability.

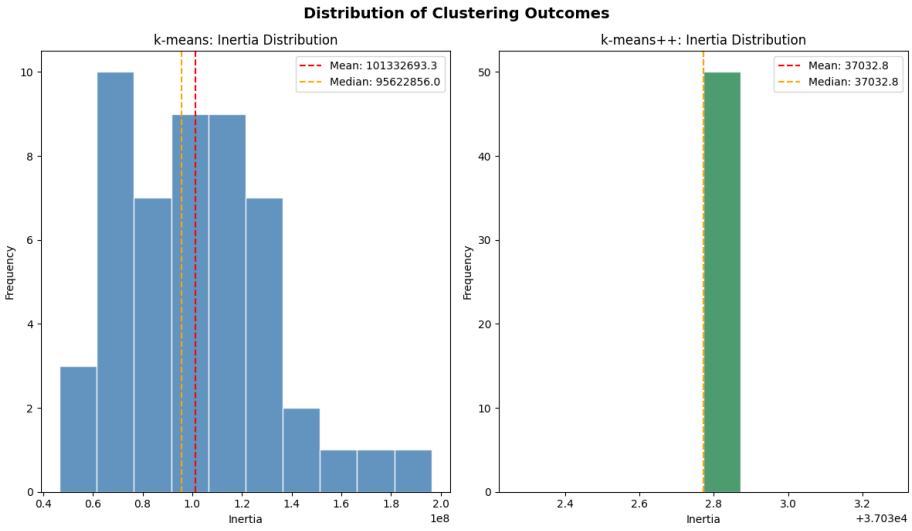

Figure 1: Inertia mean comparison

The histogram 1 is one of the most informative results, since the algorithms are run with $k = 25$ which is the number of the true centres of the dataset. It can be observed that k-means++ only has one column, which means that almost all the trials converge to the same result, showing a variability of approximately zero. On the other hand basic k-means shows high variability.

3.2 Real world datasets

	Mean ϕ			CV	
k	KM	KM++	Impr.	KM	KM++
10	22011441.96	15549998.52	29.63%	0.35	0.07
25	7454792.88	4853164.63	34.90%	0.21	0.03
50	4351503.28	2586656.37	40.56%	0.13	0.02

Table 2: Results on the cloud dataset.

	Mean ϕ			CV	
k	KM	KM++	Impr.	KM	KM++
10	169650559.52	89717921.29	47.12%	0.00	0.14
25	147024909.03	17572663.01	88.05%	0.11	0.08
50	135759250.89	6769380.49	95.01%	0.22	0.06

Table 3: Results on the spambase dataset

Tables 3.2 and 3.2 show the results on the cloud dataset and spambase

dataset.

In the cloud dataset results we can see that k-means++ lowers the inertia mean and has lower variability. It can be also observed that the improvements are less significant than the ones in the synthetic dataset. This is because the cloud dataset is a real-world dataset where the clusters are not perfectly separated.

In the spambase results we can observe even more pronounced results in comparison with the cloud dataset. The improvement on the inertia means are more significant and k-means++ confirms to be more stable. One interesting case is $k = 10$. In this case k-means results more stable than k-means++ with a coefficient of variation of 0.00, but with an average inertia almost twice than the one obtained by k-means++. This means that k-mean is stably stuck in a poor local minimum.

4 Extensions

The extension section describes two more experiments on k-means and k-means++ algorithms. During these experiments the algorithms have been run for 50 trials for k values of 10, 20, 25, 40, 50, to give more refined results.

4.1 High dimensionality

	Mean ϕ			CV	
k	KM	KM++	Impr.	KM	KM++
10	30759625419.52	30266255488.24	1.60%	0.03	0.00
20	15664668524.20	9940201473.15	36.54%	0.16	0.03
25	11161481808.01	140303237.75	98.74%	0.22	2.74
40	3077100316.63	61547665.63	98.00%	0.83	0.00
50	1390803367.28	61266609.90	95.59%	1.31	0.00

Table 4: Results for high dimensionality dataset

The results for high dimensionality dataset have been obtained by generating a synthetic dataset like the previous one but with different parameters: the number of features has been increased from 15 to 1000 and the standard deviation of the Gaussians has been increased to 5 to generate more overlapping and emphasize the results. For comparison, the algorithms were also tested on a low dimensional version of the same data set (15 dimensions instead of 1000), which led to the percentage improvement in inertia mean reported in 4.1.

The results show how k-means++ still improves inertia in high dimensionality environment while decreasing variability. In particular it can be seen that the improvement on inertia is smaller in high dimensionality with respect to low dimensionality, for k smaller than 25(optimum). For k greater than 25 it is slightly greater.

k	Impr.
10	5.89%
20	53.03%
25	97.96%
40	96.43%
50	93.09%

Table 5: KM++ Mean ϕ percentage improvement with respect to KM for the low dimensionality dataset

4.2 Overlapping clusters

For the overlapping clusters case the algorithms have been tested on a synthetic dataset generated like the previous but with standard deviation of 2 and hypercube side of 30.

	Mean ϕ			CV	
k	KM	KM++	Impr.	KM	KM++
10	1281311.20	1246112.84	2.75%	0.05	0.04
20	606229.01	513792.46	15.25%	0.12	0.12
25	426436.28	279982.18	34.34%	0.22	0.21
40	200671.18	145018.09	27.73%	0.33	0.08
50	150194.26	138308.20	7.91%	0.15	0.00

Table 6: Results for overlapping clusters datasets

Table 4.2 shows the results for overlapping clusters dataset. It can be seen how k-means++ still has an advantage over k-means in terms of inertia, with a peak at 25 (optimum), but the improvement is reduced with respect to the well separable cases.

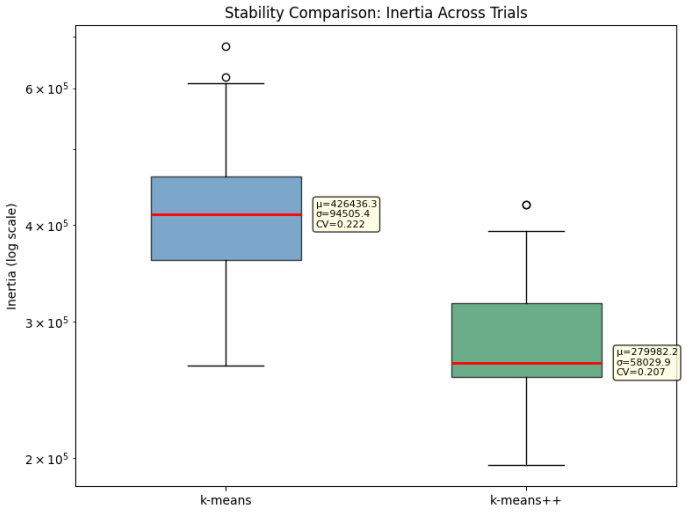

Figure 2: Boxplots of k-means and k-means++ inertia for $k = 25$

Figure 2 shows how k-means and k-means++ inertia are closer than the well separated case and k-means++ inertia presents more variability.

5 Conclusion

In conclusion the results, both on synthetic and real world datasets, shows that k-means++ algorithm reduces the inertia mean and the inertia variability over several trials. This show how standard k-means is sensible to initialization, while k-means++ can produce good clustering solution more often.

The experimental results are consistent with those reported by Arthur and Vassilvitskii, confirming that careful seeding significantly improves both the quality and the stability of k-means clustering.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

References

[1] D. Arthur and S. Vassilvitskii. k-means++: The Advantages of Careful Seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2007.